INT161 · Basic Backend Development

Mock Exam Simulation

ทำตามลำดับที่อาจารย์พาซ้อมสอบ: รับไฟล์ นำเข้า database อ่านใบงาน รันตัวตรวจ และทำข้อ 1 สดๆ

What You Get | Load the Database | Read the Spec | Run the Checker | Question 1 Live | From Sample to Exam | Rules and Rehearsal | อภิธานศัพท์

01

What You Get

อาจารย์พาซ้อมเหมือนซ้อมหนีไฟ ตอนนี้สบาย ๆ เพื่อให้วันจริงไม่มีอะไรแปลกใจ คุณต้องโหลดสี่อย่าง

`Your spec`

ไฟล์ .md ที่มีรหัส ชื่อ และ secret key ของคุณ เปิดด้วยโหมด preview

`init-database.sql`

ไฟล์เดียวทุกคนใช้ร่วมกัน สร้าง database และใส่ข้อมูลให้

`The template`

โปรเจกต์ที่มี layer ให้แล้ว ตั้งชื่อตามตัวคุณ

`tests/ folder`

ตัวตรวจ ก๊อปทั้งโฟลเดอร์ไปไว้ในโปรเจกต์

- ใบงานเป็นของแต่ละคน มีรหัสและ secret key อาจารย์เทียบกับตารางของเขาตอนตรวจ
- อาจารย์เปิดด้วย preview ใน VS Code ใช้ preview จะอ่านตารางและ code ง่าย

- ระบบตัวอย่างคือร้านขายมือถือ: brands กับ products วันสอบหัวข้ออาจเปลี่ยน แต่คาดว่าจะเป็นสองตารางและ one-to-many หนึ่งเส้น

> ⚠ **ชื่อคุณไม่อยู่ที่นี่**
> หน้านี้ใช้คนสมมติ: Somchai Jaidee รหัส 67000000000 key AB12cd ให้เปลี่ยนเป็นค่าจากใบงานของคุณเอง ห้ามใช้ของคนอื่น

02

Load the Database

งานแรก: นำเข้า database ถ้าขั้นนี้ไม่ผ่าน ก็สอบไม่ได้

ใบงานบอกให้รันคำสั่งนี้

```
from the spec

$ mysql -u root -p < init-database.sql
```

อาจารย์ลองบนเครื่องตัวเองต่อหน้าทุกคน และมันไม่ผ่าน

1. terminal ไม่รู้จัก mysql เพราะโฟลเดอร์ bin ของ MySQL ไม่อยู่ใน **PATH**
2. เขาหา MySQL ใน Program Files แล้วเพิ่มโฟลเดอร์ bin ใน PATH ของ user และเปิด terminal ใหม่
3. ตอนนี้เจอคำสั่งแล้ว แต่ login ยังไม่ผ่าน เขาเลยเลิกวิธีนี้
4. เขาใช้ IDE แทน คลิกขวาที่ data source เลือก Run SQL Script เลือกไฟล์ แล้ว refresh

- หลัง refresh จะเห็น mobile_distributor มีสองตาราง คือ brands กับ products ตาราง brands ของเขามี 22 แถว
- วันสอบอาจยังไม่มี **connection** สร้าง data source ไว้ก่อน
- ที่เขาย้ำ: คุณเรียน SQL มาแล้ว แค่รันสคริปต์ให้เป็น เครื่องมืออะไรที่รันไฟล์ได้ก็ใช้ได้

> 💡 **บน VS Code และ Windows**
> อาจารย์ใช้ WebStorm ส่วน VS Code ไม่มี data source ให้ใช้ MySQL Workbench หรือ cmd /c หรือ source ดูขั้นตอนใน Exam Walkthrough หัวข้อ 2

03

Read the Spec

อ่านใบงานให้ครบก่อนเขียน code ใบงานบอกอะไร ก็ทำตามนั้น

- ใส่รหัสนักศึกษาในตัวแปรใน app.js
- ทุก **route** ขึ้นต้นด้วย /api/รหัสของคุณ นี่คือ base path ที่เหลือคือ **resource**
- base path ผิด ตัวตรวจเข้าไม่ถึง ทุกข้อไม่ผ่าน
- สร้าง **endpoint** ที่ตอบข้อมูลของคุณ: GET /api/รหัสของคุณ/student-info ตอบ 200 ได้ 10 คะแนน
- อาจารย์ตรวจด้วยตารางรหัสและ key ถ้าของคุณไม่ตรง คำตอบนั้นผิด

ถัดไปคือ brands 10 คะแนน รายการตอบทุก field ของตาราง พร้อม success และ **timestamp** ขอทีละ id ได้ 200 หรือ 404 ถ้าไม่เจอ

products ใหญ่ที่สุด 65 คะแนน: รายการพร้อม page กับ **limit** ดูทีละตัวด้วย join สร้าง แก้ไข และลบ Exam Walkthrough สร้างครบทั้งหมด

04

Run the Checker

รัน **server** ที่ **port** 3000 เปิด terminal ที่สอง แล้วรัน **checker** พร้อมรหัสของคุณเอง

terminal 2

```
$ node tests/test-runner.mjs 67000000000
```

อาจารย์รันกับโปรเจกต์ที่ยังไม่ได้ทำอะไร ทุกข้อไม่ผ่าน และไม่เป็นไร มันทำให้เห็นข้อสอบ

checker output

```
  • [1. Identity] Student Identity Endpoint ... FAILED (0/10 pts)
    ↳ Reason: Expected HTTP 200, got 404
  • [2. Brands API] GET All Brands (List) ... FAILED (0/5 pts)
# ...one line per test, 12 tests
TOTAL SCORE              : 0 / 100 points
```

- ข้อที่ไม่ผ่านจะพิมพ์ Reason บอกว่าตัวตรวจคาดหวังอะไรและได้อะไร เช่น "expected 200, got 404" อ่านแล้วแก้ทีละอย่าง แล้วรันใหม่
- แต่ละข้อให้คะแนนแยก ทำได้บางส่วนก็ได้คะแนนบางส่วน
- ตัวตรวจเป็นไฟล์เดียวกันทุกคน รหัสของคุณบอกว่าจะเรียก route ไหน

นี่คือการรันรอบหลังจากรายงานตัวอย่าง สองข้อไม่ผ่าน และ Reason ชี้ว่าต้องแก้อะไร

checker output, later run

```
  • [1. Identity] Student Identity Endpoint ... FAILED (0/10 pts)
    ↳ Reason: studentId mismatch. Expected '67000000000', got '12367000000000'
  • [3. Products Query] Summary DTO Filtering (GET /products) ... FAILED (0/10 pts)
    ↳ Reason: Summary DTO missing essential fields: 'id', 'sku', 'name'
TOTAL SCORE              : 80 / 100 points
```

คะแนน 100 แบ่งแบบนี้

- Identity 10 Brands 5 + 5
- Products **Query** 35: **pagination** envelope 15, **summary DTO** 10, detail 10
- Products Modify 30: create 10, update 10, delete 10
- Standards 15: SKU ซ้ำ 409, **JSON** envelope, route ที่สะอาด ข้อละ 5

> **คำถามจากในคลาส**
> นักศึกษาถามว่าได้ 50 จาก 100 ที่นี่จะกลายเป็นกี่คะแนนในวิชา เสียงตอบในอัดไม่ชัด ให้ถามใน Teams

05

Question 1 Live

อาจารย์ทำข้อ 1 สด ไม่ต้องมี layer เป็น route เดียวใน app.js

1. ทำตัวแปรเก็บรหัสของคุณ
2. เขียน app.get ด้วย path /api/${ID}/student-info
3. ก๊อปรูปแบบ **response** จากใบงาน อย่าพิมพ์เอง วางแล้วใส่รหัส ชื่อ และ key ของคุณ
4. เขาใส่ timestamp ตายตัวก่อน ของจริงมาจาก new Date().toISOString()
5. restart server แล้วรัน checker ใหม่ ข้อ 1 ผ่าน

src/app.js

```
// 67000000000 - Somchai Jaidee
import express from 'express'
const STUDENT_ID = '67000000000'
const app = express()
// the spec says: every route starts with /api/<your id>
app.get(`/api/${STUDENT_ID}/student-info`, (req, res) => {
    res.status(200).json({
        success: true,
        data: { studentId: STUDENT_ID, name: 'Somchai Jaidee', secretKey: 'AB12cd' },
        timestamp: new Date().toISOString(),
    })
})
app.listen(3000, () => console.log('listening on port 3000'))
```

- เขาบอกตรง ๆ: ข้อสอบจริงจะบังคับให้มี layer (**service**, **repository**, **mapper**) ข้อนี้ไม่บังคับ จึงเป็นคะแนนง่าย
- หลังรันครั้งแรก ต้อง restart server ทุกครั้งที่บันทึก ยกเว้นใช้ **node** --watch

ข้อ 2 brands เขาให้นักศึกษาทำเอง: ตอบทุกอย่าง 200 เมื่อเจอ 404 เมื่อไม่เจอ ลองทำแล้วรัน checker

06

From Sample to Exam

อาจารย์บอกว่า template ข้อสอบหน้าตาเหมือนคลาสที่แล้ว: route, service, repository, mapper โปรเจกต์ตัวอย่างคลาส 8 คือแบบนั้น ที่ทำไว้สำหรับ classicmodels ตอนสอบเปลี่ยนแค่ชื่อ ไม่เปลี่ยนโครงสร้าง

Database

classicmodels → mobile_distributor
db-config.js

Key column

productCode → id
every SQL with WHERE

Mapper fields

productName, MSRP, scale → sku, name, brandId, category, price, status
product-mapper.js

Duplicate check

productLine → sku
create() in the repository

Routes

offices, productlines, products → brands, products
app.js

ตัวอย่างก็มีจุดพลาด แก้เหล่านี้ก่อนจะเชื่อมัน

Offset

ใช้ pageSize ที่ไม่ถูกจำกัด limit เกิน 100 จะข้ามแถวผิด

Typo in office-route.js

เรียก apiRespone.success ซึ่งไม่มีอยู่จริง ให้ใช้ apiResponse(res, data)

POST and PUT

ส่ง JSON เปล่า ไม่มี success และ timestamp ตัวตรวจให้คะแนน envelope

Password

db-config.js ใส่รหัสผ่านที่ไม่ใช่ของคุณไว้ ให้ใส่ของคุณ

errorHandler

ถูก register หลัง listen ใช้ได้ แต่ให้วางหลัง route และก่อน listen

pagination-middleware.js

```
// pagination-middleware.js in the class 8 sample
const limit = Math.min(parseInt(pageSize), 100)
const offset = (page - 1) * pageSize   // wrong: pageSize is not capped
// fix: use the capped limit
const offset = (page - 1) * limit
```

product-route.js

```
// product-route.js: POST and PUT in the sample send bare JSON.
// The checker wants the same envelope as every other route.
import httpStatus from 'http-status'
router.post("/", async (req, res) => {
    const result = await service.createObject(req.body)
    await apiResponse(res, result, null, httpStatus.CREATED)
})
router.put("/:id", async (req, res) => {
    const result = await service.updateObject(req.params.id, req.body)
    await apiResponse(res, result)
})
```

product-repository.js

```
// product-repository.js: duplicate check on sku, not on productLine
export async function create(data) {
    const [dup] = await db.query('SELECT id FROM products WHERE sku = ?', [data.sku])
    if (dup.length > 0) {
        throw new AppError(`SKU ${data.sku} already exists`, httpStatus.CONFLICT)
    }
    const [result] = await db.query('INSERT INTO products SET ?', [data])
    return findOne(result.insertId)
}
```

- POST **body** ใช้ชื่อแบบ **snake_case** เช่น brand_id และ ram_gb ถ้าตรงกับชื่อ column INSERT … SET ? รับ body ได้เลย
- **Express** 5 ส่ง error ที่ **throw** ใน **async** route ไปที่ error handler เอง ไม่ต้องมี try และ catch ในทุก route

> ⚠ **เช็ค column ของคุณ**
>
> ชื่อ table และ column ใน snippet มาจาก field ในใบงาน เทียบกับ schema จริงใน database ของคุณก่อนก๊อป

07

Rules and Rehearsal

กติกาของวันสอบจากอาจารย์ และสิ่งที่เขาขอให้ทำต่อ

- ข้อสอบ offline รัน **npm** install ครั้งเดียวตอนมีเน็ต แล้วปิดเน็ต เปิดกลับมาถือว่าทุจริต
- ปีก่อนมีนักศึกษาสอบไม่ได้เลย เขาจึงจัดซ้อมครั้งนี้
- คำถามเรื่องสอบให้โพสต์ใน Teams ของวิชา เพื่อให้ทุกคนเห็นคำตอบ เขาไม่ตอบข้อความส่วนตัวใน Line
- สัปดาห์หน้าไม่มีเรียน คุณมีเวลาสองสัปดาห์ซ้อม

ซ้อมตามลำดับนี้ จับเวลาด้วย

1. แตก zip ของ template เช็คว่าโฟลเดอร์ไม่ซ้อนกันสองชั้น
2. npm install แล้วปิดเน็ต

3. นำเข้า init-database.sql แล้ว refresh เห็นสองตาราง
4. ตั้ง db-config.js และรหัสของคุณ เช็คว่า server รันที่ port 3000
5. รัน checker หนึ่งรอบเพื่อดูข้อสอบ แล้วทำข้อ 1
6. brands แล้วค่อย products รัน checker หลังทุก endpoint
7. zip โฟลเดอร์งานโดยไม่มี node_modules ตั้งชื่อ รหัสของคุณ_exam.zip

> **ซ้อมให้ครบรอบ**
> ในคลาสไม่ได้ส่งจริง วันสอบต้องส่ง ซ้อมขั้น zip ด้วย

08

อภิธานศัพท์

async
เริ่มงานที่ช้า แล้วทำงานอื่นต่อ และถูกเรียกกลับเมื่อเสร็จ แทนที่จะรอ

body
ข้อมูลที่ request พกมาหลัง header สำหรับ API มักเป็น JSON

checker
script tests/test-runner.mjs เรียก server ที่รันอยู่ ตรวจแต่ละ endpoint ตามใบงาน และพิมพ์คะแนน

connection
ช่องทางที่เปิดอยู่ระหว่างโปรแกรมกับ database ต้องมีก่อนส่ง SQL ได้

DTO
สำเนาของ entity ที่ปรับรูปสำหรับ API มีเฉพาะ field ที่ client ต้องการ ในรูปแบบที่ตกลงกัน

endpoint
method กับ path หนึ่งคู่ที่ API ตอบ เช่น GET /api/subjects

Express
web framework ขนาดเล็กของ Node.js เพิ่ม routing และ middleware บน http module ที่มีมาให้

JSON
JavaScript Object Notation: รูปแบบข้อมูลที่เป็นข้อความ คนและโปรแกรมอ่านได้

limit
จำนวนรายการต่อหนึ่งหน้า client ส่งมาเป็น limit และ server กำหนดเพดาน

mapper

class ที่แปลง entity เป็น DTO เพื่อให้การแปลงอยู่ที่เดียวและใช้ซ้ำได้

node

runtime ที่รัน JavaScript นอก browser ทำให้ใช้ภาษาเดียวกันเขียน back end ได้

npm

Node Package Manager: ติดตั้ง library เช่น express ลงใน node_modules

pagination

ส่งรายการขนาดใหญ่ทีละหน้า เพื่อให้ response เล็กและเร็ว

PATH

ค่าตั้งของ Windows ที่ระบุโฟลเดอร์ซึ่งใช้ค้นหาคำสั่งที่พิมพ์ ถ้า bin ของ MySQL ไม่อยู่ในนี้ จะไม่รู้จักคำสั่ง mysql

port

เลขที่บอกว่าโปรแกรมไหนบนเครื่องควรได้รับข้อความ web server ใช้ 80 เป็นค่าเริ่มต้น

Query

ส่วนของ URL หลัง ? เช่น page=2&size=10 ใช้กรองหรือแบ่งหน้ารายการ

repository

layer ที่อ่านเขียน database ไม่มี business rule และไม่มี HTTP

resource

สิ่งที่ API ให้เข้าถึง เช่น product หรือ customer ในวิชานี้ตรงกับ table ใน database

response

สิ่งที่ server ส่งกลับ: status code, header และ body

route

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

server

โปรแกรม (หรือเครื่องที่รันมัน) ที่รอรับ request แล้วตอบกลับ

service

layer ที่เก็บ business rule เรียก repository และไม่รู้เรื่อง HTTP

snake_case

คำต่อกันด้วย underscore เช่น user_name ใช้บ่อยกับคอลัมน์ใน database ไม่ใช้กับ URI

summary

DTO สองขนาด: summary มีไม่กี่ field สำหรับรายการ detail มีครบสำหรับรายการเดียว

throw

โยน error ออกมา โยนได้เฉพาะ object ของ Error

timestamp

ค่าวันที่และเวลา คอลัมน์อย่าง last_update ใส่เวลาปัจจุบันให้เองได้